

AiSIO FIEMAP 논쟁 기술 분석: Christoph Hellwig vs. Samsung

출처: SNIA SDC StorageAI 2026 — AiSIO: Orchestrating Storage I/O
Across CPUs and Accelerators

분석 대상: 강연 중 14:34 및 Q&A 26:35에 기록된 Christoph Hellwig의 이의
제기와 Samsung 측 답변

1. 논쟁의 경위

Samsung 연구원이 AiSIO의 오픈소스 컴포넌트를 소개하는 중, 청중석에서 Christoph Hellwig가 발언을 끊으며 이의를 제기했다. 이후 Q&A에서 더 상세한 비판을 이어갔다.

Christoph Hellwig의 발언 (14:34, 강연 중 인터럽트)

“is a proper way to do it and I told for two years to multiple members in your team how to do it and it's really upsetting that you can keep publishing kind of dangerous”

자막이 일부 잘렸으나 문맥상 다음과 같이 읽힌다: “그렇게 하는 것이 올바른 방법이고, 저는 2년 동안 당신 팀의 여러 구성원에게 어떻게 해야 하는지 말했습니다. 위험한 것을 계속 발표하는 것은 정말 실망스럽습니다.”

Christoph Hellwig의 발언 (26:35, Q&A)

“debugging tool. It has different content for different file systems. It's a massive risk for uh cost and corruption due to concurrent activity. That's why I told everyone including your boss two years ago, you need to expand the PFS block layout layouts that have all the mechanisms to deal with that including relocation mechanism mechanism for defending clients”

번역: “그것은 디버깅 도구입니다. 파일 시스템마다 내용이 다릅니다. 동시 활동으로 인한 손상 위험이 매우 큼니다. 그래서 제가 2년 전에 당신 상사를 포함한 모든 사람에게 말했습니다. PFS 블록 레이아웃을 확장해야 한다고, 재배치 메커니즘을 포함해 클라이언트를 보호하는 모든 메커니즘을 갖추도록.”

Samsung 측 답변 (27:14)

“that was more a comment than a than a question and uh I think we’ll take that into consideration.”

“그건 질문이라기보다 의견이었고, 고려해보겠습니다.” 그리고 추가로, 팀에서 PNF(pNFS)와 flex files도 관리 레이어 후보로 검토 중이며 F_MAP_AP만 사용하는 것이 아니라고 설명했다.

2. 쟁점의 핵심: F_MAP_AP (FIEMAP)을 DMA에 사용하는 것의 위험성

2.1 F_MAP_AP / FS_IOC_FIEMAP이란?

FS_IOC_FIEMAP은 Linux 커널의 ioctl로, 파일의 논리적 바이트 범위를 물리적 블록 주소(extent)로 매핑해 반환한다. struct fiemap과 struct fiemap_extent 배열로 결과를 돌려주며, 각 엔트리에 물리적 디스크 블록 번호가 담긴다.

AiSIO의 Extend Access Library는 이 인터페이스(또는 유사한 SEEK_DATA/SEEK_HOLE + 커널 내부 경로)를 통해 파일의 물리적 LBA를 캐시해둔다. GPU가 스토리지에서 직접 데이터를 읽을 때 파일시스템을 거치지 않고 이 캐시된 주소로 NVMe 커맨드를 발행한다.

Christoph Hellwig가 “디버깅 도구”라고 표현한 것은 기술적으로 정확하다. FIEMAP은 원래 e2fsck, filefrag, xfs_bmap 같은 **진단·분석 유틸리티** 용도로 설계되었다. 파일시스템은 이 인터페이스에 동시성 보장을 제공하지 않는다.

2.2 TOCTOU 경쟁 조건 (Time-of-Check to Time-of-Use Race)

FIEMAP이 반환하는 물리적 블록 주소는 **호출 시점의 스냅샷**이다. 그 이후 어떤 작업이든 블록 주소를 바꿀 수 있으며, AiSIO의 캐시와 실제 레이아웃 사이에 불일치가 발생한다.

시간 →

t0: FIEMAP 호출 → LBA 1000 반환, 캐시에 저장

t1: 커널, 파일 블록을 LBA 2000으로 이동

t2: GPU가 LBA 1000으로 NVMe Read 발행
결과: 엉뚱한 데이터 읽기, 또는 다른 파일의 데이터 노출

이것이 Christoph Hellwig가 말한 “corruption due to concurrent activity”의 정체다.

2.3 파일시스템별 구체적 위험 요소

XFS - `xfs_swap_extents()`: 온라인 디프래그(`xfs_fsr`) 중 두 파일의 extent를 원자적으로 교환한다. FIEMAP 조회 후 이 작업이 실행되면 캐시된 LBA가 다른 파일을 가리킨다. - `xfs_alloc_file_space() + fallocate`: 사전 할당 확장 시 extent 재배치 가능. - XFS 온라인 `fsck(xfs_scrub)`의 extent 재배치 루틴도 투명하게 블록을 옮길 수 있다.

ext4 - `ext4_move_extents()` (온라인 디프래그 ioctl): 명시적 extent 이동. - 저널링 트랜잭션 중 delayed allocation 확정 단계에서 물리 블록 번호가 결정되거나 변경될 수 있다.

btrfs - Copy-on-Write 기반 설계: 쓰기 시 항상 새 블록에 기록하고 메타데이터를 업데이트. AiSIO 캐시가 가리키는 이전 블록은 새 데이터를 담지 않는다. - 백그라운드 balance 작업이 투명하게 블록을 재배치한다.

공통 - `fallocate + FALLOC_FL_PUNCH_HOLE`: 특정 범위를 hole로 만들어 블록 반환. - `truncate + 재쓰기`: 파일 축소 후 확장 시 다른 블록 할당.

2.4 Samsung의 EBPF 완화책이 충분하지 않은 이유

AiSIO 팀이 제시한 완화책은 “F_MAP_AP 결과 캐시에 EBPF 트레이스를 붙여 파일 변경 알림을 받는다”는 것이다. 이 접근의 문제점:

① 알림 지점과 실제 블록 이동 사이의 간격

EBPF 훅이 파일 변경 이벤트를 감지하더라도, 훅 실행과 캐시 무효화 사이에 짧은 창이 존재한다. GPU가 이 창 안에 NVMe 커맨드를 발행하면 경쟁 조건이 그대로 남는다. 이는 소프트웨어 레이어에서 원자성을 보장하기 어려운 구조적 문제다.

② 감지 가능한 이벤트의 범위 한계

`inotify/fsnotify` 기반 EBPF 훅은 `write()`, `truncate()`, `unlink()` 같은 VFS 레이어 이벤트를 감지한다. 그러나 아래 사례는 VFS 이벤트 없이 물리 블록이 바뀐다: - XFS 온라인 디프래그(`xfs_fsr`): `xfs_swap_extents()`는 파일 내용이 아닌 extent 트리만 수정한다. - `xfs_scrub`의 extent 재배치: 파일 데이터는 동일, 물리 주소만 변경. - `btrfs balance`: 파일 데이터 변경 없이 블록 이동.

③ 안정적이지 않은 커널 ABI

내부 커널 함수에 EBPF 호출을 거는 것은 공식 커널 ABI에 해당하지 않는다. 커널 버전 간 함수 시그니처나 호출 경로가 바뀌면 호출이 무용지물이 된다. 커널 커뮤니티가 이를 upstream에 수용할 가능성이 낮다.

④ 사전 할당 파일에서의 안전한 외관

Samsung의 벤치마크가 “pre-allocated fixed-size files”를 사용한다고 밝혔다. 이 조건에서는 블록 재배치가 거의 일어나지 않아 문제가 드러나지 않는다. 그러나 프로덕션 환경에서 백그라운드 디프래그가 활성화된 워크로드에서는 침묵하는 데이터 손상이 발생할 수 있다.

3. Christoph Hellwig가 제안한 올바른 해법: pNFS 블록 레이아웃

Christoph Hellwig가 언급한 “PFS block layout with relocation mechanisms for defending clients”는 pNFS(Parallel NFS) 블록 레이아웃 프로토콜을 가리킨다.

3.1 pNFS 블록 레이아웃의 작동 방식 (RFC 5663)

pNFS는 메타데이터 서버(MDS)와 데이터 서버(DS)를 분리한다. 클라이언트가 직접 스토리지에 접근하려면 MDS로부터 레이아웃 그랜트(layout grant)를 받아야 한다.

클라이언트 → MDS: LAYOUTGET 요청

MDS → 클라이언트: 레이아웃 그랜트 (파일 범위 → 물리 블록 매핑 포함)

클라이언트: 그랜트 범위 내에서 직접 스토리지 DMA 수행

MDS가 블록 재배치 필요 시: 클라이언트에게 LAYOUTRECALL 발행

클라이언트: 진행 중인 I/O 완료 후 레이아웃 반환(LAYOUTRETURN)

MDS: 안전하게 블록 재배치 수행

이 프로토콜의 핵심은 레이아웃 그랜트가 살아있는 동안 파일시스템이 해당 extent를 재배치할 수 없다는 강제적 보장이다. FIEMAP과 달리 스냅샷이 아닌 임대(lease) 개념이다.

3.2 Flexible File Layout (RFC 8435)

더 현대적인 flex file layout은 pNFS 위에 다중 미러링, I/O 페일오버, 레이아웃 위임(delegation)을 추가한다. 클라이언트 DMA 중 재배치가 필요한 경우 레이아웃 recall → I/O 일시정지 → 재배치 완료 → 새 레이아웃 발급의 명시적 핸드셰이크로 일관성을 보장한다.

3.3 왜 이것이 “defending clients”인가

Christoph Hellwig의 “defending clients” 표현은 pNFS의 recall 메커니즘을 가리킨다. 파일시스템이 클라이언트에게 레이아웃을 회수(recall)하기 전에 클라이언트가 I/O를 완료하거나 중단할 수 있도록 보장한다. 클라이언트는 회수 응답 전까지 유효한 레이아웃을 가지고 있음이 보장된다. 이것이 FIEMAP+EBPF 방식과의 근본적인 차이다: FIEMAP은 사후 알림(post-hoc notification)을 시도하고, pNFS는 사전 동의(prior consent)를 요구한다.

4. Samsung 측 입장의 평가

Samsung 발표자는 두 가지 답변을 제시했다:

① “F_MAP_AP를 단독으로 사용하지 않는다”

이는 Hellwig의 비판을 직접 반박하지 않는다. FIEMAP이 I/O 경로의 일부로 사용되는 한, 레이아웃 보장 없이 DMA 주소를 결정하는 근본 문제는 남는다.

② “PNF와 flex files도 관리 레이어 후보로 검토 중”

이것이 핵심적인 양보다. Samsung 팀도 pNFS 계층이 필요함을 인식하고 있다는 의미다. 그러나 현재 구현에서 pNFS 레이아웃 관리 없이 FIEMAP 기반 캐시를 사용하는 것은 Hellwig의 지적대로 위험하다.

③ “벤치마크 파일은 사전 할당된 고정 크기”

이것은 문제를 회피하는 실험 설계다. 실제 데이터 로더 워크로드에서는 파일 생성·삭제·수정이 빈번하고 백그라운드 디프래그가 활성화되어 있다.

5. 결론

Christoph Hellwig의 비판은 기술적으로 타당하다. 요약하면:

쟁점	FIEMAP + EBPF (Samsung 현재 방식)	pNFS 블록 레이아웃 (Hellwig 권고)
동시성 보장	없음 (스냅샷)	있음 (lease/recall)
블록 재배치 시 안전성	경쟁 조건 발생 가능	recall 핸드셰이크로 보장
파일시스템 호환성	파일시스템마다 동작 상이	프로토콜 수준 추상화
커널 upstream 수용 가능성	낮음 (내부 ABI 의존)	높음 (표준 RFC 기반)

쟁점	FIEMAP + EBPF (Samsung 현재 방식)	pNFS 블록 레이아웃 (Hellwig 권고)
실험 조건 한계	고정 사전 할당 파일에 서만 안전	일반 워크로드에서 안전

Samsung의 설계 목표(“interoperability over replacement”, “accelerators as first-class citizens”)는 가치 있지만, 현재 FIEMAP 기반 extent 캐시는 프로덕션에서 사용하기 위한 안전 보장을 결여하고 있다. Hellwig가 2년 전부터 이 문제를 제기했음에도 구현이 바뀌지 않은 것은, Samsung 팀이 기술적으로는 문제를 인식하면서도 pNFS 레이어 통합의 복잡성을 피하고 있음을 시사한다.

AiSIO가 실제 운영 환경에서 사용되려면 pNFS 블록 레이아웃 또는 그에 상응하는 레이아웃 임대 메커니즘이 필요하다. EBPF 알립 기반 완화책은 연구 프로토타입 수준에서만 허용 가능한 접근이다.